

Information Technology Institute (ITI)

Ministry of Communications and Information Technology — Egypt

Backend Node.js

Track Assignments

Generative AI & Prompt Engineering

Practical Assignments

Summer Training 2026

Confidential — For ITI Students Only

Task 1: AI-Assisted API Debugging

Objective

Learn how to use AI tools to **debug** broken Node.js/Express API code by writing progressively better prompts and critically evaluating AI suggestions.

Scenario

You just joined a team as a junior backend developer. Your tech lead sent you a small Express API with **5 intentional bugs**. Your job is to find and fix them. You are allowed to use AI tools to help — but you must **document your process** and **understand every fix**.

Buggy Code to Debug

```
const express = require('express');
const app = express();

// Bug 1: Missing middleware
app.post('/api/users', (req, res) => {
  const { name, email } = req.body; // undefined!
  if (!name || !email) return res.status(400).send('Missing fields');
  res.status(201).json({ name, email });
});

// Bug 2: Wrong HTTP method
app.post('/api/users/:id', (req, res) => {
  res.json({ id: req.params.id, name: 'Ahmed' });
});

// Bug 3: Incorrect status code
app.delete('/api/users/:id', (req, res) => {
  res.status(200).json({ message: 'Created successfully' });
});

// Bug 4: Port issue
app.listen('three thousand', () => console.log('Running'));
```

Requirements

1. Copy the buggy code into a new project and try to run it. Document the errors you see.
2. Write your **first prompt** to any AI tool asking for help debugging the code.
3. Review the AI's response. Identify at least **one thing the AI got right** and **one thing it missed or got wrong**.
4. Write an **improved prompt** that is more specific (mention the exact error, the endpoint, what you expect).
5. Write a **final professional prompt** using at least one technique (RTF, Few-shot, or CoT).
6. Fix all bugs **yourself** based on your understanding. Add a comment above each fix explaining what was wrong.

Expected Deliverables

- Fixed source code file (**server.js**) with comments above each fix.
- A document containing your **3 prompts** and the **AI responses** (screenshots or copy-paste).
- A **comparison table**: what each prompt produced vs. what was actually correct.
- Reflection answers.

AI Usage Rules

Allowed:

- Ask AI to explain error messages.
- Ask AI to identify potential bugs.
- Ask AI what a specific line of code does.
- Iterate and improve your prompts.

NOT Allowed:

- Paste the code and say 'fix everything' then submit the result.
- Submit AI output without understanding each fix.
- Skip writing the 3 progressive prompts.

Task 2: Prompt-Driven Express Middleware

Objective

Learn how to use AI to **understand and build** Express middleware by writing prompts that teach you the concept, then applying it yourself.

Scenario

Your team lead asks you to add three middleware functions to an existing Express API: a **request logger**, an **API key authentication** middleware, and an **input validation** middleware. You have never written custom middleware before.

Requirements

1. Write an **initial prompt** asking AI to explain what Express middleware is and how it works. Save the response.
2. Write an **improved prompt** asking AI to show you a practical example of a logging middleware with explanation of each line.
3. Write a **professional prompt** using RTF: 'Act as a senior Node.js developer. Explain the middleware pattern in Express with a real-world analogy. Show a complete example of authentication middleware using API keys. Format as commented code with a summary table of the middleware lifecycle.'
4. Now **write all 3 middleware functions yourself** — do NOT copy from AI. Use what you learned.
5. Test each middleware and document the results with screenshots.
6. Ask AI to **review your code**. Document what AI suggested improving and whether you agree or disagree (with reasons).

Expected Deliverables

- Source code with 3 middleware functions (logger, auth, validation).
- Your 3 prompts and AI responses.
- Testing screenshots showing each middleware working.
- AI code review feedback + your agree/disagree analysis.
- Reflection answers.

AI Usage Rules

Allowed:

- Ask AI to explain middleware concepts.
- Ask AI to review your code after you write it.
- Ask AI for analogies to understand the concept.
- Use AI to check for edge cases.

NOT Allowed:

- Copy middleware code directly from AI.

Task 3: AI-Powered Code Review & Refactoring

Objective

Learn how to use AI as a **code reviewer** — evaluate its suggestions critically, detect when AI is wrong, and refactor code with professional practices.

Scenario

You inherited a legacy Express + MongoDB CRUD API for a 'Products' collection. The code works but is poorly written: no error handling, no input validation, callback hell, hardcoded values, and no separation of concerns. Your task is to refactor it professionally.

Legacy Code Provided

```
app.get('/products', function(req, res) {
  db.collection('products').find().toArray(function(err, results) {
    res.send(results);
  });
});

app.post('/products', function(req, res) {
  db.collection('products').insertOne({
    name: req.body.name, price: req.body.price
  }, function(err, result) {
    res.send('Added');
  });
});
```

Requirements

1. Identify at least **5 code quality issues** in the legacy code yourself before asking AI.
2. Write your **first prompt** asking AI to review the code. Compare AI's findings with yours.
3. Write an **improved prompt** asking AI for a refactoring plan with priorities (high/medium/low).
4. Write a **professional prompt** asking AI to suggest refactored code following MVC pattern, async/await, proper error handling, and input validation.

5. Refactor the code **yourself** using AI suggestions as guidance (not copy-paste). Create a proper folder structure (routes/, controllers/, models/).
6. After refactoring, ask AI to **compare your old and new code**. Document what AI says improved and what could still be better.

Expected Deliverables

- Your initial 5 issues list (before AI).
- Refactored project with MVC structure.
- 3 prompts with AI responses.
- Before/After comparison document.
- Reflection answers.

AI Usage Rules

Allowed:

- Ask AI to identify code smells.
- Ask AI for refactoring strategies.
- Ask AI to compare old vs new code.
- Ask AI about best practices for project structure.

NOT Allowed:

- Let AI refactor the entire codebase and submit it.
- Ignore issues AI missed.
- Submit without testing the refactored code.